Lab 2: Automatic Emergency Braking

I. Learning Goals

- Using the `LaserScan` message in ROS 2
- Instantaneous Time to Collision (iTTC)
- Safety critical systems

II. Lab Policies

This is an individual assignment. You may discuss the assignment with other students; however, you must write and submit your own code. If you discuss the assignment with others, you are required to disclose the names of any students you worked with in the Canvas comments section when submitting your assignment.

Artificial Intelligence Disclosure:
If you used any generative AI tools to complete this assignment, you must disclose that you used them and how in the canvas comment section for the submission.

You may use generative AI in this course for brainstorming ideas, understanding concepts and documentation, debugging assistance, code explanation, refactoring suggestions, and improving clarity or organization of written or technical material.

You may NOT use generative AI to generate complete solutions, core project implementations, or competition code without substantial student contribution and understanding; during exams, in-class evaluations, or project demonstrations, unless explicitly permitted; or by uploading private assignment or project code to external AI services.

Please refer to the course syllabus for the complete policy on AI usage.

III. Overview

The goal of this lab is to develop a safety node for the race cars that will stop the car from collision when travelling at higher velocities. We will implement Instantaneous Time to Collision (iTTC) using the `LaserScan` message in the simulator.

For different commonly used ROS 2 messages, they are kept mostly the same as in ROS 1. You can use the command

```
Shell
 ros2 interface show <msg_name>
```

 to see the definition of messages. Note for messages that are not installed by default by the distro we use in our container, you'll have to first install it for this to work.
You will be using the f1tenth_gym_ros to test your node. Be sure to follow the instructions to set up and configure the simulator.

The `LaserScan` Message

LaserScan message contains several fields that will be useful to us. You can see detailed descriptions of what each field contains in the API. The one we'll be using the most is the `ranges` field. This is an array that contains all range measurements from the LiDAR radially ordered. You'll need to subscribe to the `/scan` topic and calculate iTTC with the LaserScan messages.

The `Odometry` Message

Both the simulator node and the car itself publish Odometry messages. Within its several fields, the message includes the cars position, orientation, and velocity. You'll need to explore this message type in this lab.

The `AckermannDriveStamped` Message

You've already used AckermannDriveStamped in the previous lab. It will be the message type that we'll use throughout the course to send driving commands to the simulator and the car. In the simulator, you can stop the car by sending an `AckermannDriveStamped` message with the `speed` field set to 0.0.

IV. The TTC Calculation

Time to Collision (TTC) is the time it would take for the car to collide with an obstacle if it maintained its current heading and velocity. We approximate the time to collision using Instantaneous Time to Collision (iTTC), which is the ratio of instantaneous range to range rate calculated from current range measurements and velocity measurements of the vehicle.
As discussed in the lecture, we can calculate the iTTC as:

$$iTTC = \frac{r}{\{-\dot{r}\}_+}$$

where r is the instantaneous range measurements, and r˙ is the current range rate for that measurement. And the operator {}+ is defined as {x}+ = max(x,0). The instantaneous range r to an obstacle is easily obtained by using the current measurements from the `LaserScan` message. Since the LiDAR effectively measures the distance from the sensor to some obstacle. The range rate r˙ is the expected rate of change along each scan beam. A positive range rate means the range measurement is expanding, and a negative one means the range measurement is shrinking. Thus, it can be calculated in two different ways. First, it can be calculated by mapping the vehicle's current longitudinal velocity onto each scan beam's angle by using vxcosθi. Be careful with assigning the range rate a positive or a negative value. The angles could also be determined by the information in `LaserScan`messages. The range rate could also be interpreted as how much the range measurement will change if the vehicle keeps the current velocity and the obstacle remains stationary. Second, you can take the difference between the previous range measurement and the current one, divide it by how much time has passed in between (timestamps are available in message headers), and calculate the range rate that way. Note the negation in the calculation this is to correctly interpret whether the range measurement should be decreasing or increasing. For a vehicle travelling forward towards an obstacle, the corresponding range rate for the beam right in front of the vehicle should be negative since the range measurement should be shrinking. Vice versa, the range rate corresponding to the vehicle travelling away from an obstacle should be positive since the range measurement should be increasing. The operator is in place so the iTTC calculation will be meaningful. When the range rate is positive, the operator will make sure iTTC for that angle goes to infinity. After your calculations, you should end up with an array of iTTCs that correspond to each angle. When a time to collision drops below a certain threshold, it means a collision is imminent.

V. Automatic Emergency Braking with iTTC

For this lab, you will make a Safety Node that should halt the car before it collides with obstacles. To do this, you will make a ROS 2 node that subscribes to the `LaserScan` and `Odometry` messages. It should analyze the `LaserScan` data and, if necessary, publish an `AckermannDriveStamped` with the `speed` field set to 0.0 m/s to brake. After you've calculated the array of iTTCs, you should decide how to proceed with this information.

You'll have to decide how to threshold, and how to best remove false positives (braking when collision isn't imminent). Don't forget to deal with `inf`s or `nan`s in your arrays.
To test your node, you can launch the sim container with `kb_teleop` set to `True` in `sim.yaml`. Then in another `bash` session inside the sim container, launch the `teleop_twist_keyboard` node from `teleop_twist_keyboard`package for keyboard teleop. It should already be installed as a dependency of the simulator. After running the simulation, the keyboard teleop, and your safety node, use the reset tool for the simulation and drive the vehicle towards a wall.
Note the following topic names for your publishers and subscribers:

- `LaserScan`: /scan
- `Odometry`: /ego_racecar/odom, specifically, the longitudinal velocity of the vehicle can be found in `twist.twist.linear.x`
- `AckermannDriveStamped`: /drive

VI: Deliverables and Submission

You can implement this package in either python or c++, and skeleton code has been provided in the zip file attached to this assignment page. Use the docker container and workspace you setup for the simulator.

Deliverable: To submit your work, upload a video to the Canvas page for this assignment of yourself driving into the wall as described in part IV. Additionally, upload the node code to the canvas page as a file attachment. A portion of the submissions will be peer reviewed by course staff to ensure that the code is academically honest(i.e. video is not faked, code implements AEB).